Raport z realizacji zadania rekrutacyjnego

PWR Racing Team

Contents

1 Wstęp **2**

1.1 Użyte narzędzia . . . 2

2 Zadanie 1 **4**

2.1 Algorytm Zachłanny (Greedy Clustering) . . . 4

2.1.1 Zasada działania algorytmu . . . 4

2.1.2 Złożoność obliczeniowa i optymalizacje . . . 4

2.1.3 Implementacja . . . 6

2.1.4 Edge casey . . . 7

2.1.5 Problemy . . . 7

2.2 Zaawansowana Redukcja Przestrzenna (Spatial Hashing z DSU) . . . 7

2.2.1 Zasada działania algorytmu . . . 7

2.2.2 Wyznaczanie centroidów i finalnych identyfikatorów . . . 8

2.2.3 Złożoność obliczeniowa i optymalizacje . . . 8

2.2.4 Implementacja . . . 10

2.2.5 Edge casey . . . 10

2.2.6 Problemy . . . 11

2.3 Wyznaczanie Otoczki Wypukłej (Monotone Chain (algorytm Andrew)) . . . 11

2.3.1 Zasada działania algorytmu . . . 11

2.3.2 Złożoność obliczeniowa i optymalizacje . . . 11

2.3.3 Implementacja . . . 12

2.3.4 Edge casey . . . 12

2.3.5 Problemy . . . 13

2.3.6 Zasada działania algorytmu . . . 13

2.3.7 Złożoność obliczeniowa i optymalizacje . . . 13

2.3.8 Implementacja . . . 13

2.3.9 Edge casey . . . 14

2.3.10 Problemy . . . 14

2.4 Wyniki działania obu algorytmów . . . 14

3 Zadanie 2 **15**
3.1 Co wiemy z treści i regulaminu? . . . 15
3.1.1 Geometria toru i parametry przestrzenne (D 4.1.2, D 4.1.5) . . . 15
3.1.2 Układ współrzędnych i warunki brzegowe (D 4.1.6) . . . 15
3.1.3 Weryfikacja fizyczna w oparciu o pas jazdy (Test spójności) . . . 15
3.2 Analiza porównawcza i charakterystyka błędów pomiarowych sensorów . . . 16
3.3 Z czym pracujemy? . . . 16
3.3.1 Charakterystyka źródeł danych . . . 17
3.4 Cel i założenia . . . 19
3.5 Dane . . . 20
3.5.1 Wczytanie i podsumowanie . . . 20
3.5.2 Surowe trajektorie . . . 21
3.6 Analiza błędów czujników (podpunkt 2.1) . . . 22
3.6.1 Sensor 2 (rzadki): odbicie osi Y, outliery, szum . . . 22
3.6.2 Sensor 1 (gęsty): dryf . . . 22
3.6.3 Charakter błędu: residuum czujnik rzadki − gęsty . . . 23
3.6.4 Podejrzewane źródła nieprawidłowości . . . 24
3.7 Metoda (podpunkt 2.2) . . . 25
3.7.1 Model sygnału . . . 25
3.7.2 Algorytm (filtr komplementarny typu “error-state”) . . . 25
3.7.3 Implementacja . . . 25
3.8 Wyniki i walidacja . . . 29
3.8.1 Trajektoria po fuzji . . . 29
3.8.2 Przebiegi w czasie . . . 30
3.8.3 Implementacja . . . 31
3.8.4 Edge casey . . . 33
3.8.5 Problemy . . . 33

1 Wstęp

1.1 Użyte narzędzia

- **CLion** (JetBrains) – Środowisko programistyczne (IDE) wykorzystane do implementacji algorytmów i struktury danych w języku C++.
- **RStudio** – Środowisko pracy z językiem R, służące do analizy danych pomiarowych oraz zarządzania dokumentacją projektu
- **C++ (Standard C++)** – Język programowania użyty do implementacji wydajnych algorytmów redukcji przestrzennej (Spatial Hash, Greedy) oraz metod wyznaczania otoczki wypukłej (Graham Scan).
- **CMake** – System automatyzacji budowania i zarządzania konfiguracją projektu C++.

- **R Markdown & LaTeX (MiKTeX)** – Narzędzia do składu tekstu i automatycznego generowania sprawozdania technicznego w formacie PDF.
- **Biblioteka ggplot2 (R)** – Narzędzie graficzne wykorzystane do tworzenia czytelnych wykresów diagnostycznych oraz wizualizacji porównawczych punktów.

2 Zadanie 1

2.1 Algorytm Zachłanny (Greedy Clustering)

2.1.1 Zasada działania algorytmu

Algorytm opiera się na iteracyjnym łączeniu punktów, których odległość euklidesowa mieści się w zadanym progu odległości *threshold* (*eps*).

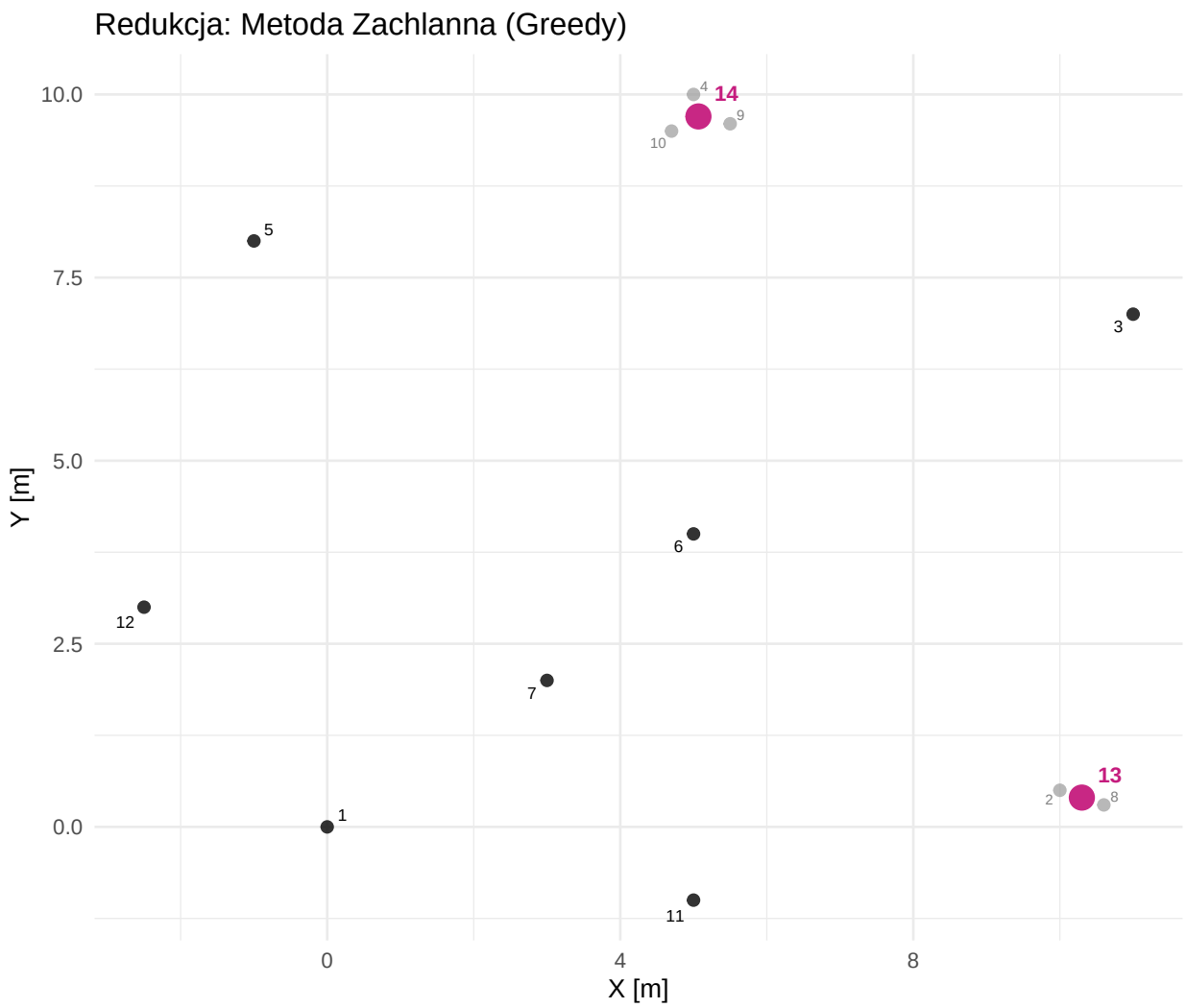
Proces przebiega w następujących krokach:

1. **Inicjalizacja:** Dla każdego punktu inicjalizowana jest lista jego oryginalnych identyfikatorów (`originalIds`), początkowo zawierająca wyłącznie jego własne `id`.
2. **Pętla iteracyjna fuzji (`while`):**
   - W każdej iteracji przeszukiwana jest para nierozłączonych jeszcze punktów.
   - Jeśli odległość między punktem bieżącym *current* a jego sąsiadem $currentPoints[j]$ spełnia warunek bliskości:
     $$\Delta x^2 + \Delta y^2 \leq threshold^2$$
     punkty te kwalifikują się do fuzji.
3. **Średnia ważona pozycji:** Nowa pozycja połączonego klastra wyznaczana jest jako średnia ważona, uwzględniająca liczbę pierwotnych punktów składających się na oba klastry:
   $$x_{new} = \frac{x_1 \cdot N_1 + x_2 \cdot N_2}{N_1 + N_2}$$
   Zapewnia to stabilność pozycji centroidu i zapobiega „pływaniu" wykrytych obiektów.
4. **Scalanie identyfikatorów:** Listy oryginalnych ID obu obiektów są łączone w jedną, a sąsiad oznaczany jest jako pochłonięty (`merged[j] = true`).
5. **Warunek stopu:** Pętla powtarza się dopóki w danej iteracji uda się dokonać jakiegokolwiek połączenia.

2.1.2 Złożoność obliczeniowa i optymalizacje

Pod względem teoretycznym klasyczny algorytm zachłanny charakteryzuje się wysoką złożonością czasową:

- **W najgorszym przypadku:** $\mathcal{O}(N^3)$ (wynikającą z wielokrotnego wykonywania przeszukiwań N^2 w pętli `while` aż do stabilizacji klastrów).
- **W przestrzeni pamięciowej:** $\mathcal{O}(N)$ na przechowywanie struktur wektorowych.


Redukcja: Metoda Zachlanna (Greedy)
10.0
7.5
5.0
2.5
0.0
Y [m]
0
4
8
X [m]
14
4
9
10
5
3
6
12
7
13
2
8
1
11

2.1.3 Implementacja

```
std::vector<Point> currentPoints = points;

// Inicjalizacja: każdy punkt początkowo zawiera sam siebie w originalIds
for (auto& pt : currentPoints) {
    if (pt.originalIds.empty()) {
        pt.originalIds = {pt.id};
    }
}

bool mergedAny = true;

while (mergedAny) {
    mergedAny = false;
    std::vector<Point> nextPoints;
    // Które łączymy
    std::vector<bool> merged(currentPoints.size(), false);

    for (size_t i = 0; i < currentPoints.size(); ++i) {
        if (merged[i]) continue;

        Point current = currentPoints[i];

        for (size_t j = i + 1; j < currentPoints.size(); ++j) {
            if (!merged[j]) {

                double distSq = calculateDistanceSquared(current, currentPoints[j]);

                // Sprawdzamy odległość
                if (distSq <= threshold * threshold) {

                    // Średnia ważona
                    double sizeCurrent = static_cast<double>(current.originalIds.size());
                    double sizeNeighbor = static_cast<double>(currentPoints[j].originalIds.size());
                    double totalCount = sizeCurrent + sizeNeighbor;

                    current.x = (current.x * sizeCurrent + currentPoints[j].x * sizeNeighbor) / tota
                    current.y = (current.y * sizeCurrent + currentPoints[j].y * sizeNeighbor) / tota

                    // Łączymy listy oryginalnych ID
                    current.originalIds.insert(
                        current.originalIds.end(),
                        currentPoints[j].originalIds.begin(),
                        currentPoints[j].originalIds.end()
                    );

                    merged[j] = true;
                    mergedAny = true;
                }
            }
        }
        nextPoints.push_back(current);
```

```
        }
        currentPoints = nextPoints;
    }
```

2.1.4 Edge casey

- Pusta lista wejściowa
- Jeden punkt
- Efekt domina

2.1.5 Problemy

Ze wzgledu na intuicyjność tego rozwiązania nie napotkałem żadnych problemów.

2.2 Zaawansowana Redukcja Przestrzenna (Spatial Hashing z DSU)

W celu przezwyciężenia ograniczeń wydajnościowych klasycznego algorytmu zachłannego, zaimplementowano wydajną metodę opartą na **siatce przestrzennej (*Spatial Hashing*)** oraz strukturze danych **Disjoint Set Union (DSU / Union-Find)**. Podejście to redukuje liczbę wymaganych porównań.

2.2.1 Zasada działania algorytmu

Algorytm dzieli przestrzeń 2D na dyskretną siatkę kwadratową, w której bok każdej komórki odpowiada progowi odległości ε (`eps`).

Proces przebiega w następujących krokach:

1. **Przypisanie do siatki:** Dla każdego punktu wyznaczane są współrzędne komórki (`cx`, `cy`) przy użyciu funkcji podłogi:
$$cx = \lfloor x/\varepsilon \rfloor, \quad cy = \lfloor y/\varepsilon \rfloor$$
Punkty umieszczane są w słowniku haszującym (`std::unordered_map`), co pozwala na natychmiastowy dostęp do obiektów w danej strefie.

2. **Przeszukiwanie sąsiedztwa i DSU:** Dla każdej niepustej komórki algorytm sprawdza wyłącznie **9 sąsiadujących komórek** (wliczając ją samą).

3. **Warunek bliskości:** Dla par punktów z komórki bieżącej oraz sąsiedniej testowany jest warunek odległości z użyciem kwadratów (bez operacji pierwiastkowania):
$$\Delta x^2 + \Delta y^2 \leq \varepsilon^2$$
Jeśli warunek jest spełniony, wywoływana jest operacja **`unite`** struktury DSU, łącząca punkty w spójny klaster.

4. **Kompresja ścieżki:** Struktura DSU zarządza relacjami w czasie niemal stałym dzięki mechanizmowi kompresji ścieżki (`path compression`) w operacji `find`.

5. **Grupowanie klastrów:** Po przetworzeniu siatki punkty są ostatecznie grupowane na podstawie wspólnego korzenia w DSU.

2.2.2 Wyznaczanie centroidów i finalnych identyfikatorów

Po zakończeniu fazy fuzji dla każdego wyznaczonego klastra wykonywane są operacje końcowe:

- **Centroid:** Pozycja nowego punktu wyliczana jest jako średnia arytmetyczna współrzędnych x i y wszystkich elementów należących do klastra.
- **Zarządzanie identyfikatorami:** Oryginalne ID punktów składowych są zbierane i sortowane rosnąco.
- Pojedyncze pachołki zachowują swoje pierwotne `id`, natomiast klastry powstałe z fuzji otrzymują nowe, sekwencyjne identyfikatory zaczynające się od wartości $maxOriginalId + 1$.

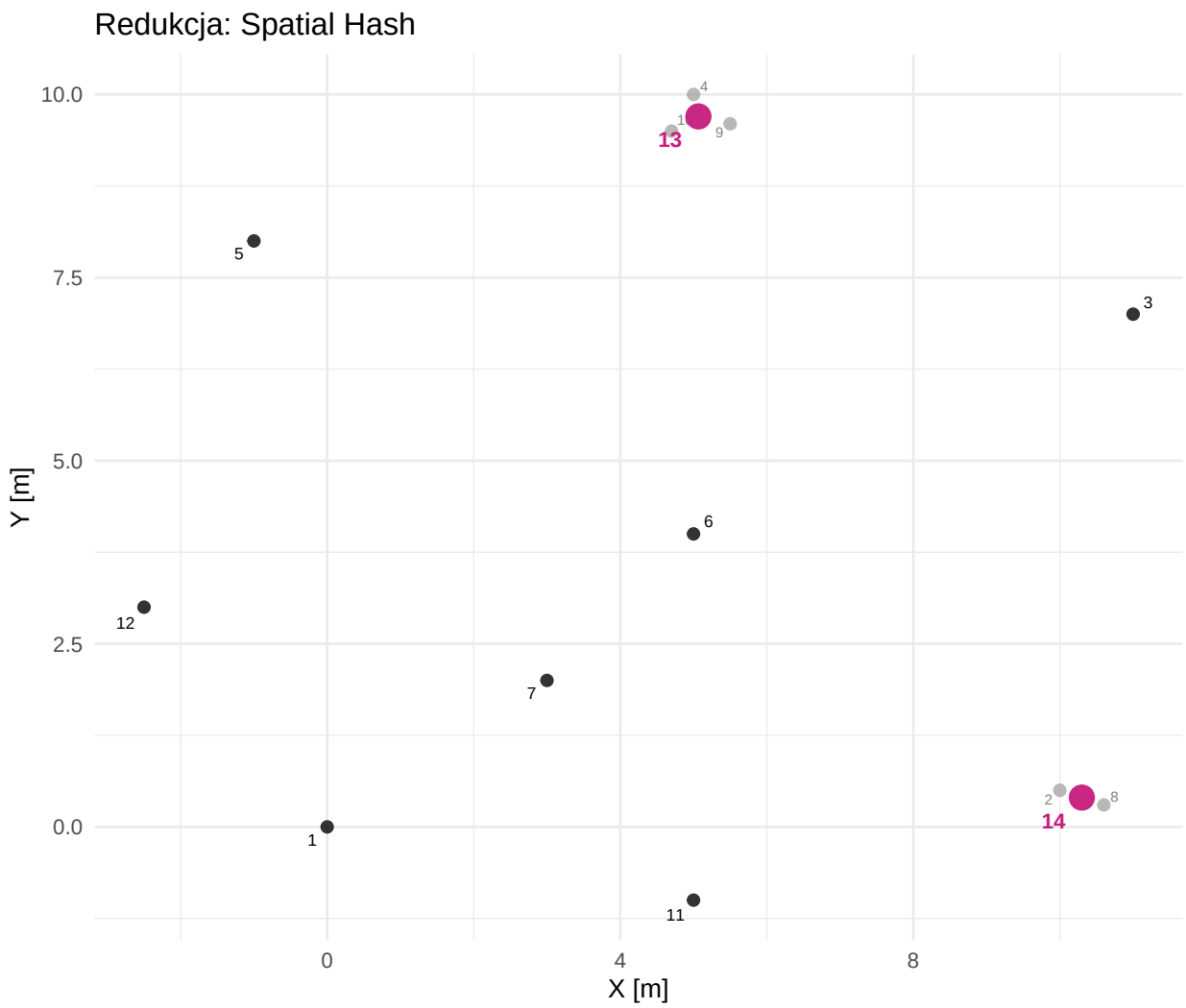
2.2.3 Złożoność obliczeniowa i optymalizacje

Pod względem teoretycznym podejście to zapewnia drastyczną poprawę wydajności:

- **W najgorszym przypadku:** $\mathcal{O}(N \cdot M)$ (gdzie M to średnia liczba punktów w sąsiadujących komórkach, co w praktyce dla równomiernie rozłożonych pachołków daje złożoność bliską liniowej $\mathcal{O}(N)$).
- **W przestrzeni pamięciowej:** $\mathcal{O}(N)$ na potrzeby struktury siatki oraz DSU.

W kodzie C++ zastosowano dodatkowe optymalizacje niskopoziomowe:

- **Użycie wspólnej funkcji kwadratu odległości (`calculateDistanceSquared`):** Całkowite wyeliminowanie kosztownych operacji `std::sqrt` i `std::pow` podczas testów sąsiedztwa w pętlach siatki.
- **Wydajne kontenery STL:** Wykorzystanie `std::unordered_map` do szybkiego haszowania komórek przestrzennych.
- **Bezpieczne przekazywanie danych:** Szerokie stosowanie stałych referencji (`const auto&`) eliminujących zbędne kopiowanie obiektów w pamięci. ## Wyznaczanie Otoczki Wypukłej (Convex Hull)


Redukcja: Spatial Hash
10.0
7.5
5.0
2.5
0.0
Y [m]
0
4
8
X [m]
4
1
9
13
5
3
6
12
7
2
8
14
1
11

2.2.4 Implementacja

```
/ Inicjalizacja oraz podział siatki na komórki
        int n = points.size();
        DisjointSet dsu(n);
        std::unordered_map<CellKey, std::vector<int>, CellHash> grid;

        // Przypisanie do siatki
        for (int i = 0; i < n; ++i) {
            int cx = static_cast<int>(std::floor(points[i].x / eps));
            int cy = static_cast<int>(std::floor(points[i].y / eps));
            grid[{cx, cy}].push_back(i);
        }

        // Sprawdzanie sąsiedztwa i DSU
        for (const auto& pair : grid) {
            const CellKey& cell = pair.first;
            const auto& currentCellPoints = pair.second;

            for (int dx = -1; dx <= 1; ++dx) {
                for (int dy = -1; dy <= 1; ++dy) {
                    CellKey neighborCell = {cell.x + dx, cell.y + dy};
                    auto it = grid.find(neighborCell);
                    if (it != grid.end()) {
                        const auto& neighborPoints = it->second;

                        for (int p1Idx : currentCellPoints) {
                            for (int p2Idx : neighborPoints) {
                                if (p1Idx >= p2Idx) continue;

                                double distSq = calculateDistanceSquared(points[p1Idx], points[p2Idx]);

                                // Porównujemy kwadrat odległości z kwadratem progu
                                if (distSq <= eps * eps) {
                                    dsu.unite(p1Idx, p2Idx);
                                }
                            }
                        }
                    }
                }
            }
```

2.2.5 Edge casey

- Eps = 0 (błąd dzielenia przez 0), dla bardzo dużych eps, wszystkie punkty łączą się w jeden
- W skrajnych przypadkach możliwość zakfalifikowania/ braku kfalifikacji przez błędy numeryczne
- Pusty wektor / 1 punkt
- 2 punkty
- Wszystkie punkty identyczne

2.2.6 Problemy

Ze wzgledu na intuicyjność tego rozwiązania nie napotkałem żadnych problemów.

- Brak napotkanych problemów, trzeba było jedynie przysiaść do algebry z 1 roku żeby zrozumieć, ale fajny algorytm.

2.3 Wyznaczanie Otoczki Wypukłej (Monotone Chain (algorytm Andrew))

2.3.1 Zasada działania algorytmu

Otoczka wypukła wyznacza najmniejszy wielokąt wypukły, który zawiera wszystkie zredukowane punkty (centroidy pachołków) w przestrzeni dwuwymiarowej.

Proces przebiega w następujących etapach:

1. **Sortowanie punktów:** Zbiór wszystkich zredukowanych pachołków jest sortowany najpierw niemalejąco według współrzędnej x, a w przypadku jej równości – według współrzędnej y.
2. **Budowa dolnej i górnej otoczki:** Algorytm iteracyjnie przetwarza posortowane punkty, budując najpierw otoczkę dolną, a następnie górną (zgodnie z metodą łańcucha monotonnego / *Monotone Chain*).
3. **Test orientacji (iloczyn wektorowy):** W trakcie budowy konturu sprawdzany jest kierunek skrętu dla kolejnych trzech punktów za pomocą iloczynu wektorowego:

   $$(p_2.x - p_1.x) \cdot (p_3.y - p_1.y) - (p_2.y - p_1.y) \cdot (p_3.x - p_1.x)$$

   Jeśli orientacja nie spełnia warunku wypukłości (występuje skręt w „złą stronę"), poprzedni punkt jest usuwany z budowanej otoczki.
4. **Zamknięcie konturu:** Połączenie obu ścieżek tworzy spójną, uporządkowaną sekwencję wierzchołków definiujących zewnętrzną granicę rozmieszczenia pachołków.

2.3.2 Złożoność obliczeniowa i optymalizacje

Pod względem teoretycznym proces wyznaczania otoczki charakteryzuje się bardzo korzystną efektywnością:

- **W czasie:** $\mathcal{O}(N \log N)$, gdzie N to liczba zredukowanych punktów (główny narzut wynika z początkowego sortowania współrzędnych).
- **W przestrzeni pamięciowej:** $\mathcal{O}(N)$ na przechowywanie struktur wierzchołków tworzących wielokąt.

2.3.3 Implementacja

```
// Sortowanie punktów
std::vector<Point> convexHull(std::vector<Point> points) {
    std::sort(points.begin(), points.end(), [](const Point& a, const Point& b) {
        if (a.x != b.x) return a.x < b.x;
        return a.y < b.y;
    });

    // Usunięcie powtarzających się punktów
    points.erase(
        std::unique(points.begin(), points.end(), [](const Point& a, const Point& b) {
            return a.x == b.x && a.y == b.y;
        }),
        points.end()
    );

    if (points.size() <= 1) {
        return points;
    }

    // Budowa dolnej części otoczki
    std::vector<Point> lower, upper;

    for (const Point& p : points) {
        while (lower.size() >= 2 &&
               cross(lower[lower.size() - 2], lower.back(), p) <= 0) {
            lower.pop_back();
               }
        lower.push_back(p);
    }

    // Budowa górnej części otoczki
    for (int i = static_cast<int>(points.size()) - 1; i >= 0; --i) {
        const Point& p = points[i];
        while (upper.size() >= 2 &&
               cross(upper[upper.size() - 2], upper.back(), p) <= 0) {
            upper.pop_back();
               }
        upper.push_back(p);
    }

    lower.pop_back();
    upper.pop_back();
```

2.3.4 Edge casey

- Pusty wektor / 1 punkt
- 2 punkty
- Wszystkie punkty identyczne

2.3.5 Problemy

Ze wzgledu na intuicyjność tego rozwiązania nie napotkałem żadnych problemów.

- Brak napotkanych problemów, trzeba było jedynie przysiaść do algebry z 1 roku żeby zrozumieć, ale fajny algorytm. ## Wyznaczanie Otoczki Wypukłej (Graham's Scan)

2.3.6 Zasada działania algorytmu

Otoczka wypukła wyznacza najmniejszy wielokąt wypukły, który zawiera wszystkie zredukowane punkty (centroidy pachołków) w przestrzeni dwuwymiarowej.

Proces przebiega w następujących etapach:

1. **Wybór punktu bazowego (kotwicy):** Algorytm znajduje punkt wysunięty najdalej w dół (o najmniejszej współrzędnej y, a przy remisie najbardziej w lewo), który gwarantuje przynależność do otoczki, i ustanawia go punktem odniesienia.
2. **Sortowanie kątowe (biegunowe):** Pozostałe punkty są sortowane rosnąco według kąta biegunowego względem kotwicy (przeciwnie do wskazówek zegara), z wykorzystaniem iloczynu wektorowego oraz kwadratów odległości w przypadku współliniowości.
3. **Filtrowanie punktów współliniowych:** Z posortowanej listy eliminowane są punkty pośrednie leżące na tym samym promieniu od kotwicy, pozostawiając jedynie punkt najbardziej oddalony.
4. **Skan Grahama i test orientacji (stos):** Algorytm wykorzystuje strukturę stosu, do której kolejno trafiają przefiltrowane punkty. Kierunek skrętu dla kolejnych trzech punktów weryfikowany jest za pomocą iloczynu wektorowego:

   $$(p_2.x - p_1.x) \cdot (p_3.y - p_1.y) - (p_2.y - p_1.y) \cdot (p_3.x - p_1.x)$$

   Jeśli orientacja nie spełnia warunku wypukłości (występuje skręt w „złą stronę" lub linia prosta), punkt ze szczytu stosu jest usuwany, aż do zachowania poprawnej struktury wielokąta.

2.3.7 Złożoność obliczeniowa i optymalizacje

Pod względem teoretycznym proces wyznaczania otoczki charakteryzuje się bardzo korzystną efektywnością:

- **W czasie:** $\mathcal{O}(N \log N)$, gdzie N to liczba zredukowanych punktów (główny narzut wynika z początkowego sortowania kątowego punktów względem kotwicy).
- **W przestrzeni pamięciowej:** $\mathcal{O}(N)$ na przechowywanie struktur wierzchołków, wektora przefiltrowanego oraz stosu budującego wielokąt.

2.3.8 Implementacja

```
// Algorytm
std::vector<Point> hull;
hull.push_back(filtered[0]);
hull.push_back(filtered[1]);
hull.push_back(filtered[2]);
```

```cpp
    for (size_t i = 3; i < filtered.size(); ++i) {
        while (hull.size() >= 2 && cross(hull[hull.size() - 2], hull.back(), filtered[i]) <= 0) {
            hull.pop_back();
        }
        hull.push_back(filtered[i]);
    }
```

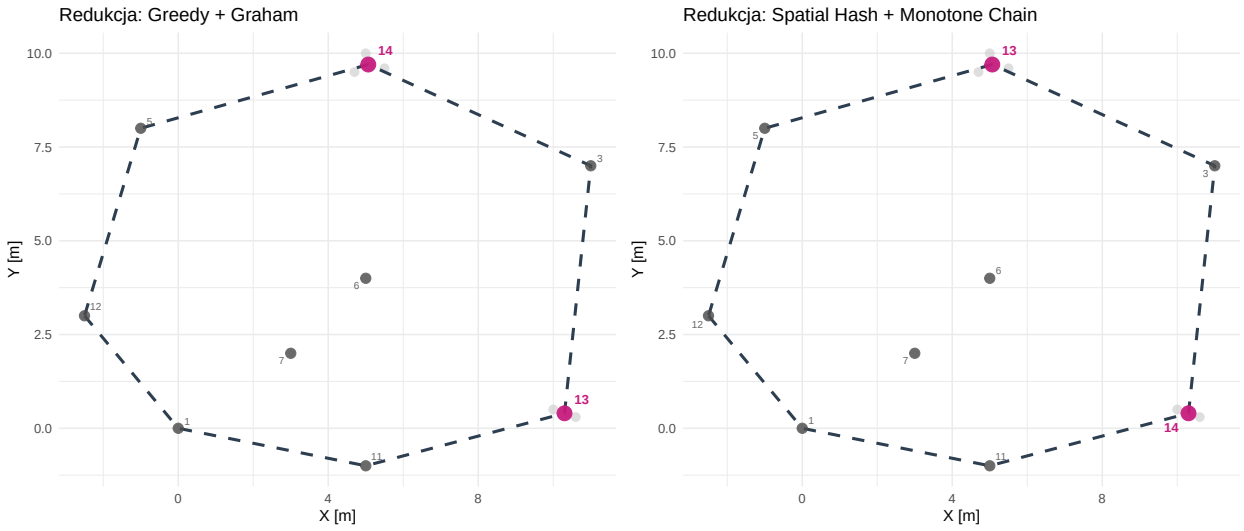
2.3.9 Edge casey

- Pusty wektor / 1 punkt
- 2 punkty
- Wszystkie punkty identyczne

2.3.10 Problemy

Ze wzgledu na intuicyjność tego rozwiązania nie napotkałem żadnych problemów.

2.4 Wyniki działania obu algorytmów



Wyniki wyjściowe uzyskane dla obu zaimplementowanych algorytmów wykazują pełną zgodność z danymi referencyjnymi zawartymi w materiałach źródłowych.

3 Zadanie 2

3.1 Co wiemy z treści i regulaminu?

3.1.1 Geometria toru i parametry przestrzenne (D 4.1.2, D 4.1.5)

- **Kształt toru:** Konkurencja odbywa się na torze w kształcie cyfry 8, składającym się z dwóch stycznych okrągów wyznaczonych przez pachołki.
- **Szerokość pasa:** Pas drogi ma stałą szerokość wynoszącą **3 m**, ograniczoną wewnętrznym okręgiem o średnicy 15.25 m (promień $r_{min} = 7.625$ m) oraz zewnętrznym okręgiem o średnicy 21.25 m (promień $r_{max} = 10.625$ m).
- **Oś geometryczna pasa:** Nominalny promień przejazdu środkiem pasa wynosi:

$$r = \frac{7.625 + 10.625}{2} = 9.125 \text{ m}$$

- **Współrzędne środków okręgów:** Przyjęto układ, w którym środki okręgów znajdują się w odległości 18.25 m od siebie, co umieszcza je w punktach $(0, \pm 9.125)$ w lokalnym układzie współrzędnych.

3.1.2 Układ współrzędnych i warunki brzegowe (D 4.1.6)

- **Linia startu/mety:** Zgodnie z punktem D 4.1.6 regulaminu, linia pomiaru czasu (start/meta) łączy środki obu okrągów, co oznacza, że współrzędna $x = 0$ leży bezpośrednio na tej linii, a okręgi stykają się w okolicach punktu $(0, 0)$.
- **Warunki początkowe i końcowe:** Informacja z treści zadania o rozpoczęciu ruchu 15 m przed linią pomiaru ($x(0) = -15.0$ m) oraz zatrzymaniu 10 m za nią ($x(33) = 10.0$ m) nie jest założeniem arbitralnym – wynika bezpośrednio z geometrii rozmieszczenia stanowiska startowego względem punktu centralnego toru.

3.1.3 Weryfikacja fizyczna w oparciu o pas jazdy (Test spójności)

- Ponieważ pojazd fizycznie porusza się wewnątrz 3-metrowego korytarza, **żaden poprawny odczyt nie może trwale opuścić wyznaczonego pasa ruchu**.
- Analiza wykresów surowych danych wykazuje, że czujniki dotknięte błędem dryfu lub inwersji przekraczają te granice (np. gęsty sensor osiąga wartości rzędu $x = -15.7$ m, wykraczając poza dopuszczalny margines). Procent punktów mieszczących się w nominalnym pasie jazdy stanowi zatem obiektywną miarę jakości każdego z sensorów przed i po fuzji.

3.2 Analiza porównawcza i charakterystyka błędów pomiarowych sensorów

3.3 Z czym pracujemy?

Analiza opiera się na dwóch plikach danych pomiarowych dostarczonych w formacie CSV: `sensor1.csv` oraz `sensor2.csv`. Każdy z plików zawiera rejestrację przejazdu w strukturze kolumnowej:

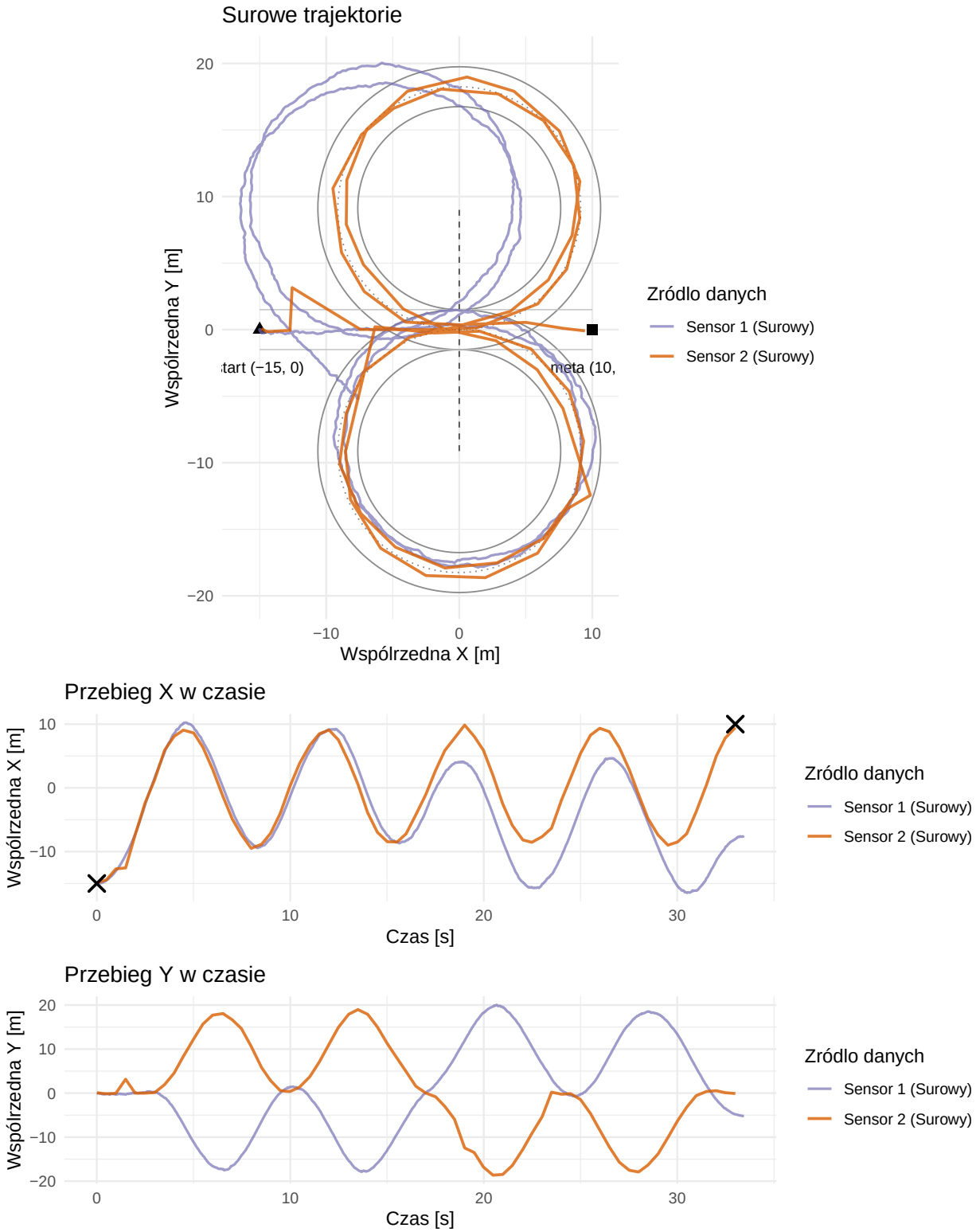
$$\text{timestamp}, x, y$$

Gdzie poszczególne pola oznaczają:

- `timestamp` – znacznik czasu w sekundach, określający moment dokonania pomiaru (dane obejmują przedział od 0.000 s do 33.002 s).
- `x` – współrzędna wzdłużna położenia środka ciężkości bolidu w metrach (dodatnie wartości oznaczają ruch w przód, w tym punkty startowe i końcowe wynikające z regulaminu).
- `y` – współrzędna poprzeczna położenia pojazdu w metrach (dodatnie wartości oznaczają przemieszczenie w lewo).

3.3.1 Charakterystyka źródeł danych

Dostarczone sensory reprezentują dwie różne technologie pomiarowe/strumienie danych o odmiennej specyfice, co wymusza zastosowanie fuzji:



1. **Sensor 1 (Strumień wysokoczęstotliwościowy, ≈ 45 Hz):**
    - *Zalety:* Charakteryzuje się wysoką gęstością prókowania, dzięki czemu dobrze oddaje dynamikę jazdy
    - *Wady:* **inwersja osi Y** (współrzędne poprzeczne są odwrócone względem rzeczywistego kierunku jazdy, co tworzy obraz lustrzany) oraz skumulowany, nieliniowy **dryf** narastający w czasie.
2. **Sensor 2 (Strumień niskoczęstotliwościowy, ≈ 2 Hz):**
    - *Zalety:* Stanowi stabilną bazę odniesienia w makroskali. Jego punkty początkowe i końcowe idealnie korelują z warunkami brzegowymi toru Skidpad ($x(0) = -15$ m oraz $x(33) = 10$ m).
    - *Wady:* Niska częstotliwość próbkowania uniemożliwia bezpośrednie odtworzenie płynnego toru jazdy (między kolejnymi pomiarami występują duże przerwy czasowe rzędu 0.5 s), a dodatkowo strumień ten zawiera sporadyczne, skokowe zakłócenia (*outliery*).

3.4 Cel i założenia

Celem jest wyznaczenie położenia środka ciężkości bolidu w **dowolnej chwili** $T \in [0, 33]$ s na podstawie dwóch błędnie działających czujników położenia (2D, widok z góry).

Założenia wynikające z treści zadania i regulaminu:

- dodatnie x to ruch w przód, dodatnie y to ruch w lewo,
- linia startu/mety to linia łącząca środki kół (regulamin D 4.1.6), więc przyjmuję $x = 0$ na tej linii; bolid startuje 15 m przed nią i zatrzymuje się 10 m za nią, czyli oczekiwane punkty graniczne to $(-15, 0)$ dla $t = 0$ i $(10, 0)$ dla $t = 33$,
- geometria toru (D 4.1.2, D 4.1.5): środki kół w odległości 18,25 m, czyli w $(0, \pm 9{,}125)$; średnica wewnętrzna 15,25 m ($R_{in} = 7{,}625$ m), zewnętrzna 21,25 m ($R_{out} = 10{,}625$ m); tor jazdy to pas **3 m** między nimi, więc oś pasa ma $R_0 = 9{,}125$ m,
- okrążenie to przejazd jednego koła od linii startu/mety do niej samej (D 4.1.6),
- kolejność kół: najpierw **prawe**, potem lewe.

3.5 Dane

3.5.1 Wczytanie i podsumowanie

Table 1: Tabela 1. Podstawowe parametry plików wejściowych.

Plik	Próbki	t od [s]	t do [s]	Mediana kroku [s]	Częstotliwość [Hz]
sensor1.csv	1339	0.000	33.447	0.025	40
sensor2.csv	67	0.008	33.002	0.501	2

3.5.2 Surowe trajektorie

Wykresy surowych trajektorii przejazdu dla każdego z czujników.

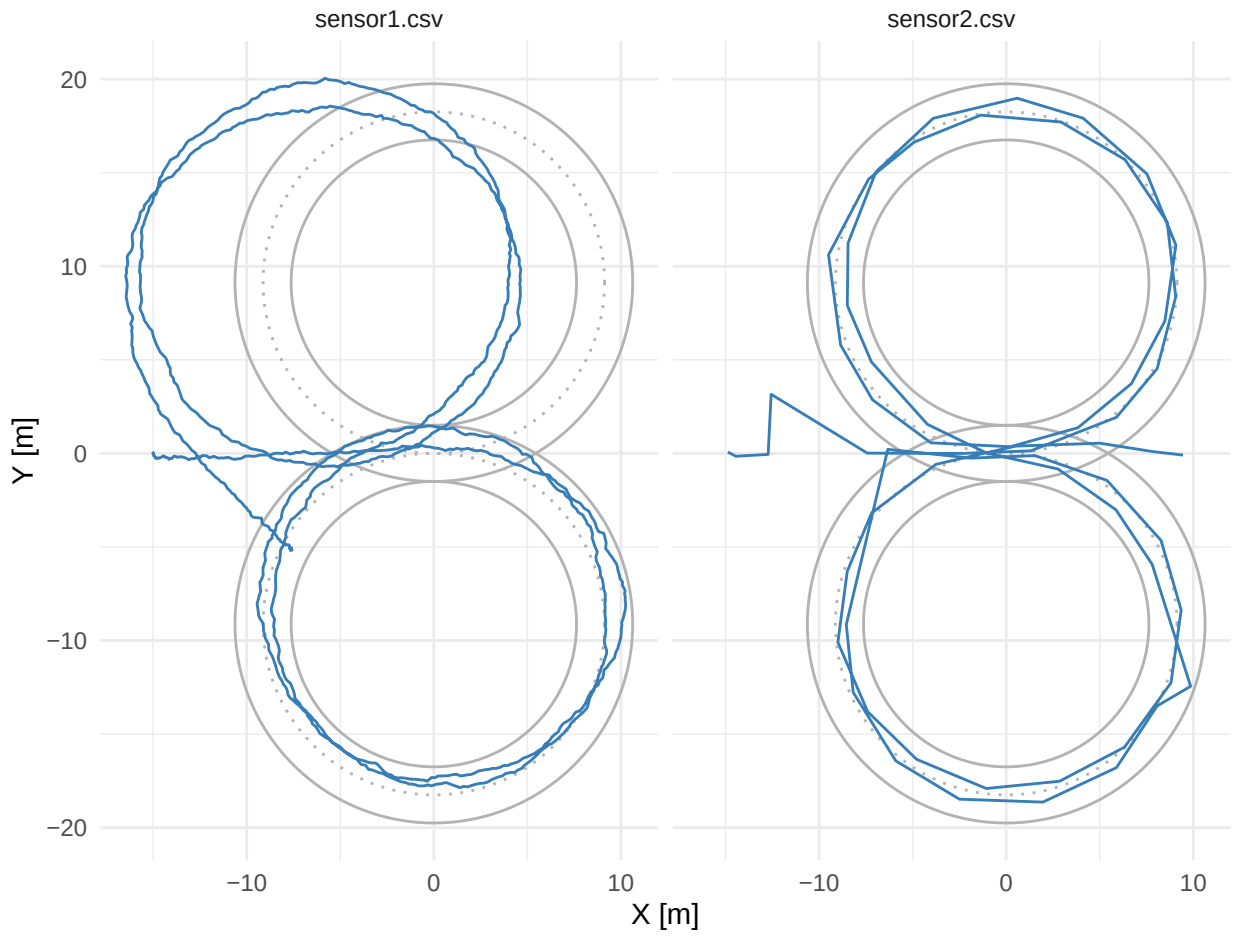


Figure 1: Rys. 1. Surowe trajektorie obu czujników (przed jakąkolwiek korektą). Linie ciągłe: granice pasa jazdy (r = 7,625 i 10,625 m), kropkowana: oś pasa (r = 9,125 m).

Obserwacje wstępne: jeden czujnik jest bardzo gęsty i gładki, drugi rzadki, kanciasty i z pojedynczymi skokami.

3.6 Analiza błędów czujników (podpunkt 2.1)

3.6.1 Sensor 2 (rzadki): odbicie osi Y, outliery, szum

- Odbicie osi Y. Na szczycie pierwszego koła Sensor 1 ma y $\approx -17{,}3$ m, a Sensor 2 y $\approx +18{,}1$ m.
- Outliery. Pojedyncze skoki o kilka metrów (np. przy t $\approx 1{,}49$ s), widoczne na Rys. 2 jako punkty odstające od gładkiej krzywej.
- Szum. Rozrzut rzędu dziesiątych części metra wokół trendu.
- Brak dryfu. Koniec toru przy ok. (9,45; $-0{,}09$), czyli ok. 0,6 m od (10, 0).

3.6.2 Sensor 1 (gęsty): dryf

- Obserwacja: zaczyna dokładnie w $(-15, 0)$, a przy t = 33 s jest w okolicy $(-8, -5)$, czyli ok. 18–19 m od (10, 0). Residuum na Rys. 2 rośnie gładko i bez skoków. Środki kół wędrują (Tabela 4). Pod koniec wychodzi poza pas jazdy (Tabela 4b).
- Wniosek: błąd zaczyna się od zera i narasta, więc to dryf, a nie stały offset.
- Hipoteza: całkowanie IMU lub żyroskopu bez korekty.

Czujnik **gęsty**: sensor1.csv (40 Hz). Czujnik **rzadki**: sensor2.csv (2 Hz). Ruch ma pasmo rzędu ułamka herca (okres jednego koła ok. 7 s), więc nawet rzadki czujnik spełnia kryterium Nyquista dla wolnych składowych. Błąd interpolacji liniowej łuku to jednak $\approx R(1-\cos(\omega h/2))$, czyli dla kroku rzadkiego czujnika rzędu dziesiątek centymetrów, a dla gęstego pomijalny. Dlatego **tor** biorę z czujnika gęstego, a rzadki posłuży do korekty.

3.6.3 Charakter błędu: residuum czujnik rzadki — gęsty

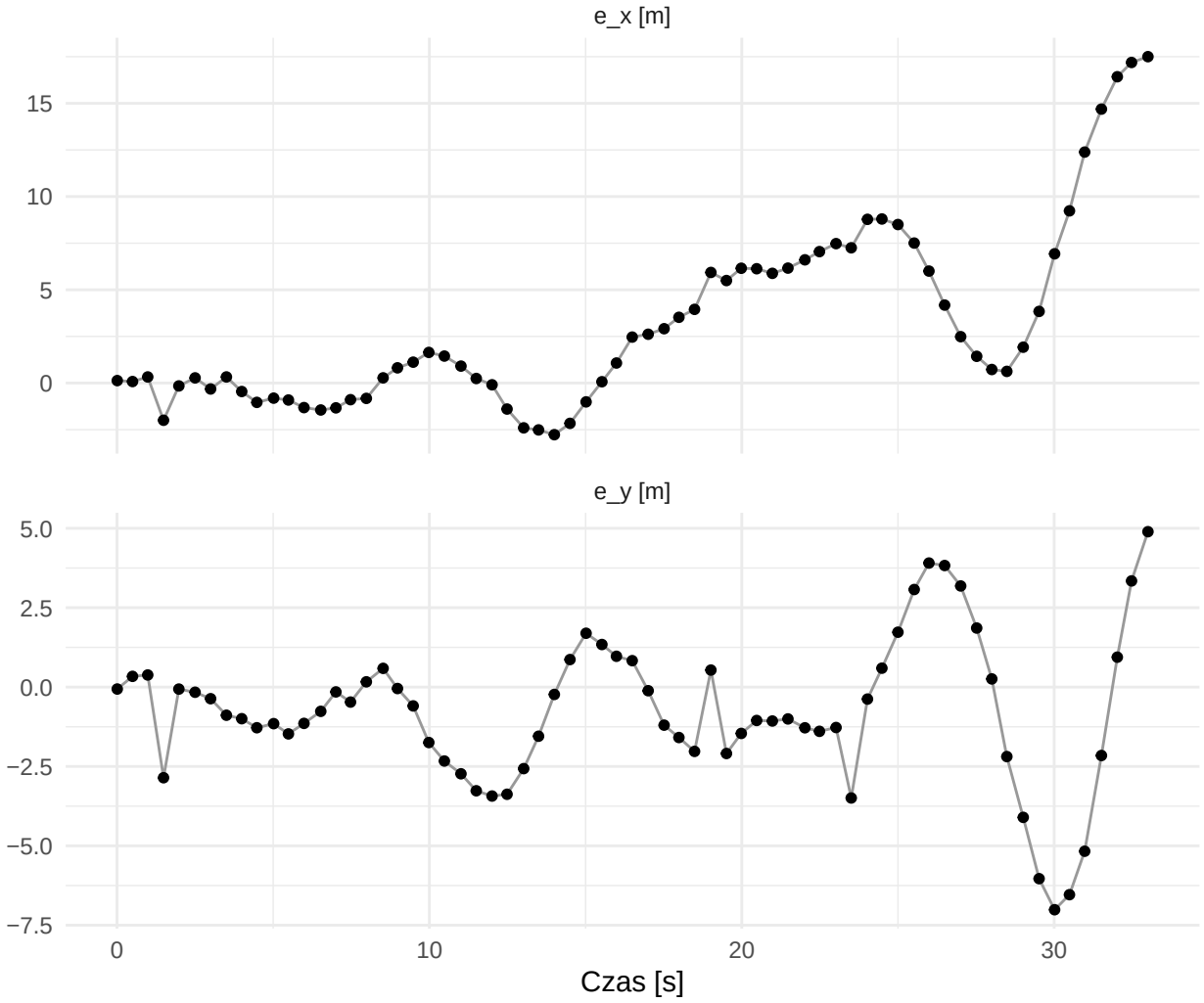


Figure 2: Rys. 2. Residuum e = rzadki − gęsty (po normalizacji osi Y). Gładka, narastająca krzywa to dryf; pojedyncze odstające punkty to outliery.

Składowa e_x: Wykazuje wyraźny, narastający w czasie trend wzrostowy – początkowo waha się w granicach kilku metrów, by w końcowej fazie przejazdu (po 28. sekundzie) gwałtownie wzrosnąć do ponad 15 m. Składowa e_y: Nie wykazuje stałego trendu wzrostowego, ale charakteryzuje się bardzo dużymi zmianami i narastającą amplitudą wahań (od −7,5 m do +5 m na końcu przejazdu).

3.6.4 Podejrzewane źródła nieprawidłowości

To **hipotezy** wnioskowane z kształtu błędów, a nie dowody.

Błąd	Obserwacja	Prawdopodobne źródło
Czujnik gęsty: dryf	gładkie, narastające rozjeżdżanie się toru; bez skoków	nieskorygowane całkowanie IMU / żyroskopu (błąd biasu żyroskopu → błąd kursu rosnący z czasem), odometria kołowa bez korekty zewnętrznej
Czujnik rzadki: outliery	pojedyncze skoki o kilka metrów, niska częstotliwość	odbiór GNSS: odbicia wielodrogowe (multipath), chwilowa utrata/zmiana satelitów
Czujnik rzadki: szum	rozrzut rzędu 0,1–0,3 m	typowy szum pomiaru pozycji GNSS
Odbicie osi Y u jednego z czujników	lustrzana kolejność kół	niespójna konwencja układu współrzędnych (np. y w prawo zamiast w lewo) lub odwrócony montaż

3.7 Metoda (podpunkt 2.2)

3.7.1 Model sygnału

$$p_f(t) = p(t) + d(t) + n_f(t), \qquad p_s(t) = p(t) + n_s(t) + o(t)$$

gdzie p to prawdziwa pozycja, d to dryf czujnika gęstego (f), n to szumy, a o to rzadkie outliery czujnika rzadkiego (s).

3.7.2 Algorytm (filtr komplementarny typu "error-state")

1. **Korekta układu odniesienia**: odwrócenie y w czujniku, który najpierw skręca w lewo (sekcja 3.2).
2. **Residuum** $e_i = p_s(t_i) - p_f(t_i)$ w chwilach pomiarów czujnika rzadkiego. Gęsty jest interpolowany liniowo. Wtedy $e \approx -d + n_s - n_f + o$, czyli estymata błędu czujnika gęstego.
3. **Filtr Hampela**: punkt i odrzucam, gdy jego odległość od mediany residuów z okna 5 próbek przekracza próg $\max(1\,\text{m},\ k \cdot 1{,}4826 \cdot \text{med}(\text{odchyleń}))$, $k = 5$. Mediana ignoruje pojedyncze skoki i nie myli wolnego trendu z anomalią (dla ciągu monotonicznego mediana to element środkowy). Stałe są heurystyką: współczynnik 1,4826 odpowiada skali rozkładu normalnego 1D, a tu mierzymy normę 2D. Dlatego w sekcji 4.6 jest test wrażliwości.
4. **Punkty skrajne**: okno Hampela jest tam jednostronne, więc mediana jest obciążona trendem. Punkt pierwszy i ostatni oceniam względem warunków brzegowych z zadania, czyli akceptuję go, jeśli leży w odległości ≤ 1 m od $(-15, 0)$ lub $(10, 0)$. Tolerancja 1 m jest mniejsza niż połowa szerokości 3-metrowego pasa wjazdu/wyjazdu (D 4.1.5), więc bolid nie mógł stać dalej od osi.
5. **Poprawka** $e(t)$: interpolacja liniowa przyjętych residuów (poza zakresem wartość skrajna).
6. **Wynik**: $\hat{p}(T) = p_f(T) + e(T)$.

Czego świadomie **nie** robię: wygładzania wyniku ani $e(t)$ (na okręgu skraca promień i tłumi składową dryfu oscylującą z częstotliwością ruchu) oraz filtru Kalmana (brak modelu pojazdu i niepewności czujników, więc macierze kowariancji byłyby dobrane arbitralnie).

3.7.3 Implementacja

3.7.3.1 1. Sformułowanie matematyczne modelu sygnałów Niech rzeczywista, ciągła pozycja środka ciężkości bolidu w czasie t będzie reprezentowana przez wektor stanu 2D:

$$p(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix}$$

Oba czujniki rejestrują ten ruch z odmienną specyfiką i zakłóceniami:

1. **Czujnik gęsty ($f_f \approx 45$ Hz):**
$$p_f(t) = p(t) + d(t) + n_f(t)$$
gdzie:
   - $d(t) = \begin{bmatrix} d_x(t) \\ d_y(t) \end{bmatrix}$ reprezentuje płynny, skumulowany **dryf** czujnika gęstego (np. z dyskretnego całkowania błędów żyroskopu/odometrii),
   - $n_f(t)$ to pomijalnie mały szum pomiarowy wysokiej częstotliwości.
2. **Czujnik rzadki ($f_s \approx 2$ Hz):**
$$p_s(t_i) = p(t_i) + n_s(t_i) + o(t_i)$$
gdzie:

- $n_s(t_i)$ to szum pomiarowy pozycji o zerowej średniej, $\mathbb{E}[n_s] = 0$,
- $o(t_i)$ stanowi rzadkie, grube błędy pomiarowe (**outliery** rzędu kilku metrów, np. z wielodrogowości sygnału GNSS).

3.7.3.2 2. Szczegółowy przebieg algorytmu krok po kroku

3.7.3.2.1 Krok 1: Korekta układu współrzędnych (Inwersja osi Y) Przed rozpoczęciem estymacji sprowadzamy dane do spójnego, prawoskrętnego układu współrzędnych. Zgodnie z regulaminem Skidpad pierwsze koło jest prawe ($y < 0$ na szczycie łuku). Dla czujnika wykazującego inwersję osi stosowana jest korekta geometryczna:

$$y_{\text{norm}}(t) = -y(t)$$

3.7.3.2.2 Krok 2: Estymacja stanu błędu (Wyznaczenie residuum e_i) Dla każdej chwili pomiarowej t_i czujnika rzadkiego interpolujemy liniowo pozycję z gęstego strumienia $\hat{p}_f(t_i) = \text{interp}(p_f, t_i)$, a następnie obliczamy wektor residuum e_i:

$$e_i = p_s(t_i) - \hat{p}_f(t_i)$$

Podstawiając modele sygnałów, otrzymujemy:

$$e_i = \Big(p(t_i) + n_s(t_i) + o(t_i)\Big) - \Big(p(t_i) + d(t_i) + n_f(t_i)\Big) = -d(t_i) + n_s(t_i) - n_f(t_i) + o(t_i)$$

Dzięki temu odejmowaniu rzeczywista, złożona trajektoria $p(t_i)$ ulega całkowitej eliminacji. Residuum e_i mierzy bezpośrednio **odwrotność dryfu** $-d(t_i)$, zanieczyszczoną jedynie szumem i wartościami odstającymi.

3.7.3.2.3 Krok 3: Detekcja i eliminacja outlierów (Filtr Hampela w 2D) Dla ciągu residuów stosujemy przesuwne okno filtru Hampela o szerokości $N = 5$ próbek ($J_i = [i-2, \dots, i+2]$):

1. Obliczamy medianę lokalną składowych w oknie:
$$\bar{e}_{x,i} = \text{mediana}_{j \in J_i}(e_{x,j}), \quad \bar{e}_{y,i} = \text{mediana}_{j \in J_i}(e_{y,j})$$
2. Wyznaczamy odchylenie euklidesowe danej próbki od lokalnej mediany:
$$\text{dev}_i = \sqrt{(e_{x,i} - \bar{e}_{x,i})^2 + (e_{y,i} - \bar{e}_{y,i})^2}$$
3. Obliczamy adaptacyjny próg odrzucenia na podstawie odchylenia bezwzględnego od mediany (MAD):
$$\text{próg} = \max\left(\text{floor},\ k \cdot 1.4826 \cdot \text{mediana}_j(\text{dev}_j)\right)$$
gdzie floor $= 1.0$ m, a $k = 5$.
4. Wyznaczamy flagę akceptacji próbki:
$$\text{accepted}_i = \begin{cases} 1 & \text{dla dev}_i \leq \text{próg} \\ 0 & \text{dla dev}_i > \text{próg} \quad \text{(outlier)} \end{cases}$$

3.7.3.2.4 Krok 4: Weryfikacja punktów brzegowych (Bramkowanie) Na brzegach dziedziny czasowej ($t = 0$ s oraz $t = 33$ s) weryfikujemy próbki skrajne względem warunków brzegowych toru:

$$\|p_s(t_1) - (-15, 0)\| \leq 1.0 \text{ m}, \quad \|p_s(t_N) - (10, 0)\| \leq 1.0 \text{ m}$$

3.7.3.2.5 Krok 5: Rekonstrukcja ciągłej funkcji poprawki dryfu $\hat{e}(t)$ Na podstawie przefiltrowanych próbek ($\text{accepted}_i = 1$) konstruujemy ciągłą w czasie funkcję poprawki dryfu za pomocą odcinkowej interpolacji liniowej:

$$\hat{e}_x(t) = \text{interp}\left(\{t_i, e_{x,i}\}_{\text{accepted}_i=1},\ t\right)$$

$$\hat{e}_y(t) = \text{interp}\left(\{t_i, e_{y,i}\}_{\text{accepted}_i=1},\ t\right)$$

3.7.3.2.6 Krok 6: Synteza końcowej trajektorii Dla dowolnej chwili czasowej $T \in [0, 33]$ s estymowana pozycja po fuzji wyliczana jest jako:

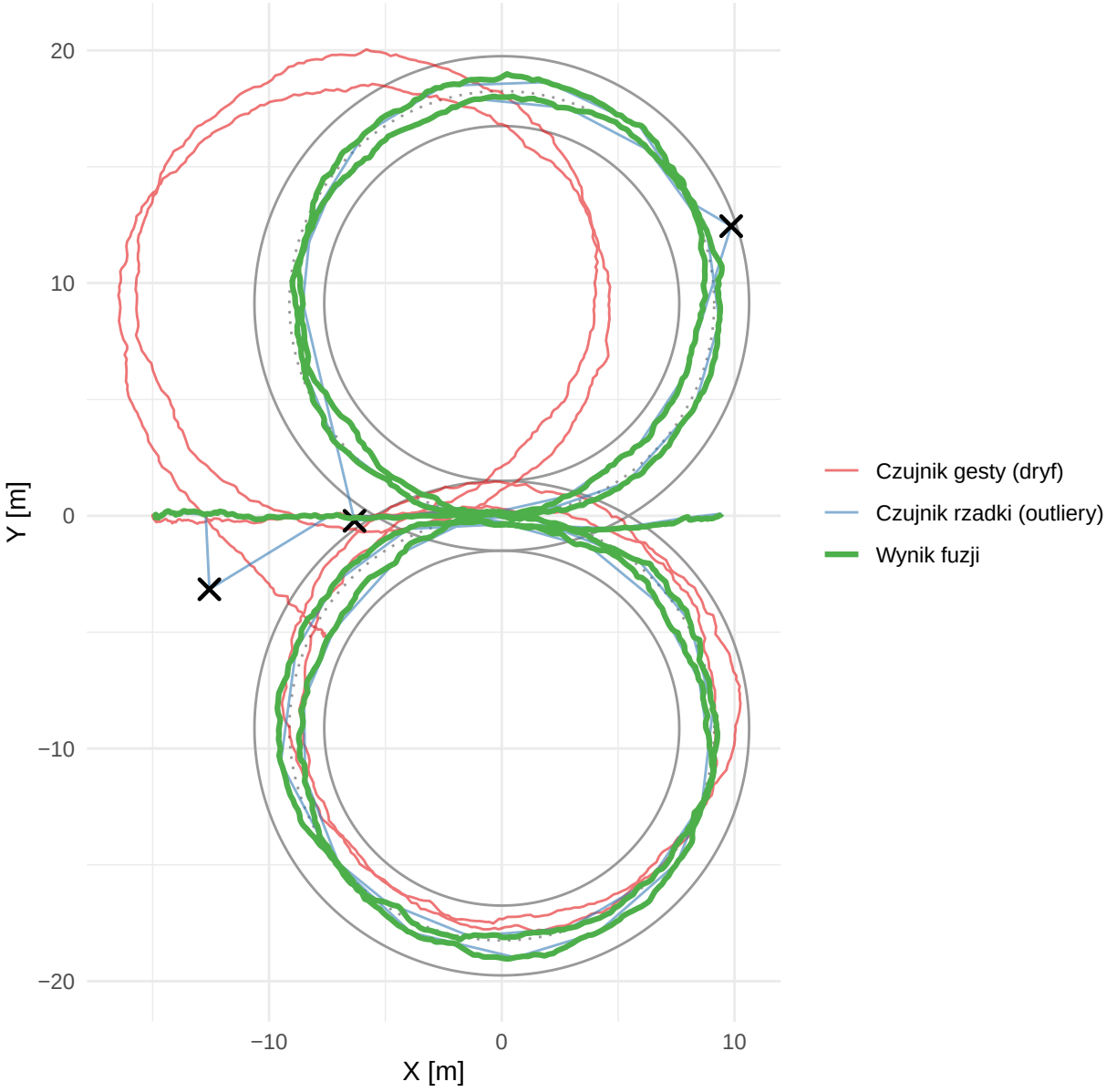
$$\hat{p}(T) = p_f(T) + \hat{e}(T) = \begin{bmatrix} x_f(T) + \hat{e}_x(T) \\ y_f(T) + \hat{e}_y(T) \end{bmatrix}$$

3.7.3.3 3. Uzasadnienie wyboru metody (Zalety inżynierskie)

1. **Komplementarność cech:** Architektura fuzji pozwala połączyć wysokie próbkowanie (45 Hz) oraz płynność czujnika gęstego z bezwzględną dokładnością i brakiem skumulowanego dryfu czujnika rzadkiego.
2. **Filtracja w przestrzeni błędów (Error-State):** Filtrowanie bezpośrednio w przestrzeni pozycji uszkodziłoby ostrość zakrętów i zmniejszyło promieniej kół. Przejście do przestrzeni błędu (e_i) izoluje powolny trend dryfu od dynamiki pojazdu, co gwarantuje pełne zachowanie geometrii przejazdu.
3. **Odporność mediany na skoki:** Wykorzystanie filtru Hampela z medianą zapewnia odporność na wartości odstające (outliery) bez zniekształcania rzeczywistego trendu poprawek.

3.8 Wyniki i walidacja

3.8.1 Trajektoria po fuzji



Widać znaczną porpawę toru ruchu i jego utrzymanie w pasie toru

3.8.2 Przebiegi w czasie

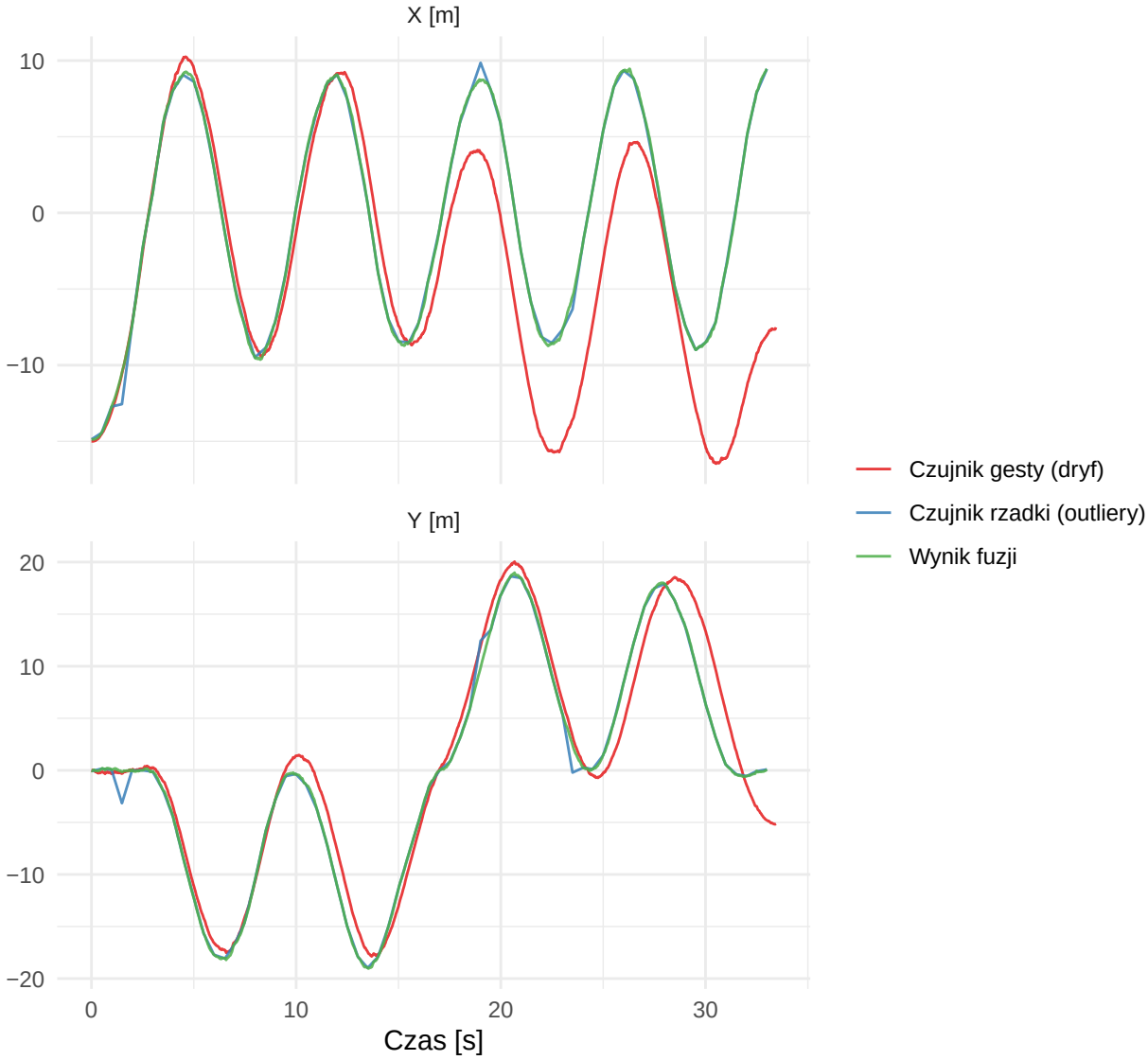


Figure 3: Rys. 4. Współrzędne X(t) i Y(t): czujniki i wynik fuzji.

Analogiczny wniosek dla przebiegów w czasie.

3.8.3 Implementacja

```
// Interpolacja liniowa, zeby móc estymwoać punkty w dowolnej chwili wyścigu
Sample interpolate(const Track& d, double T) {
    if (d.empty()) throw std::logic_error("interpolate: pusty tor");
    if (T <= d.front().t) return {T, d.front().x, d.front().y};
    if (T >= d.back().t) return {T, d.back().x, d.back().y};

    auto it = std::upper_bound(d.begin(), d.end(), T,
                               [](double val, const Sample& s) { return val < s.t; });

    const Sample& b = *it;
    const Sample& a = *std::prev(it);

    const double dt = b.t - a.t;
    const double k = (dt > 0.0) ? (T - a.t) / dt : 0.0;

    // Interpolacja liniowa położenia X oraz Y
    return {
        T,
        a.x + k * (b.x - a.x),
        a.y + k * (b.y - a.y)
    };
}

// Znalezienie, czy y>0 czy <0
int firstTurnSign(const std::vector<Sample>& d) {
    const double t0 = d.front().t;
    double best = 0.0;
    for (const Sample& s : d) {
        if (s.t - t0 > kTurnWindow) break;
        if (std::abs(s.y) > std::abs(best)) best = s.y;
    }
    return best < 0.0 ? -1 : 1;
}

// Y -> -Y
void normalizeFrame(Track& d) {
    if (firstTurnSign(d) > 0) {
        for (Sample& s : d) {
            s.y = -s.y;
        }
    }
}

// Szybkie znajdywanie mediany O(n)
double median(std::vector<double> v) {
    if (v.empty()) return 0.0;

    const size_t n = v.size();
    auto mid = v.begin() + n / 2;
    std::nth_element(v.begin(), mid, v.end());
```

```
    if (n % 2 != 0) return *mid;

    auto max_low = std::max_element(v.begin(), mid);
    return 0.5 * (*mid + *max_low);
}

Track buildCorrection(const Track& fast, const Track& slow, std::vector<char>& accepted) {
    const size_t n = slow.size();

    // Wyznaczenie błędu
    Track res(n);
    for (size_t i = 0; i < n; ++i) {
        const Sample f = interpolate(fast, slow[i].t);
        res[i] = {slow[i].t, slow[i].x - f.x, slow[i].y - f.y};
    }

    // Filtr Hampela
    std::vector<double> dev(n);
    for (size_t i = 0; i < n; ++i) {
        const size_t lo = (i >= static_cast<size_t>(kHampelHalf)) ? i - kHampelHalf : 0;
        const size_t hi = std::min(n - 1, i + kHampelHalf);

        std::vector<double> wx, wy;
        wx.reserve(hi - lo + 1);
        wy.reserve(hi - lo + 1);
        for (size_t j = lo; j <= hi; ++j) {
            wx.push_back(res[j].x);
            wy.push_back(res[j].y);
        }
        dev[i] = std::hypot(res[i].x - median(wx), res[i].y - median(wy));
    }

    const double sigma = 1.4826 * median(dev);
    const double threshold = std::max(kMinThreshold, kHampelK * sigma);

    // Przefiltrowanie próbek oraz utworzenei wektora poprawek
    accepted.assign(n, 0);
    Track corr;
    corr.reserve(n);
    for (size_t i = 0; i < n; ++i) {
        if (dev[i] <= threshold) {
            accepted[i] = 1;
            corr.push_back(res[i]);
        }
    }

    if (corr.empty()) throw std::runtime_error("Wszystkie pomiary rzadkiego sensora odrzucone");
    return corr;
}

// Tworzy fuzję ścieżek
Sample fusedAt(const Track& fast, const Track& corr, double T) {
    const Sample f = interpolate(fast, T);
```

```
    const Sample e = interpolate(corr, T);
    return {T, f.x + e.x, f.y + e.y};
}
```

3.8.4 Edge casey

- Sztywno zakodowane okno czasowe (kTurnWindow = 10.0)
- Jeśli w wolnym czujniku wystąpią 3 błędne pomiary z rzędu, mediana w oknie 5-elementowym stanie się wartością błędną (przekroczenie tzw. breakdown point mediany). W efekcie filtr Hampela uzna te błędy za poprawne dane i odrzuci punkty prawidłowe.
- Okno filtracji będzie mniejsze niż zakładane 5 punktów. Mediana i estymator σ będą wyliczane z bardzo małej próby, co drastycznie obniża skuteczność wykrywania wartości odstających.
- W warunkach brzegowych interpolate stosuje stałą ekstrapolację (zwraca pierwszą/ostatnią znaną pozycję). Na odcinkach $t \in [0, 2)$ oraz $t \in (30, 33]$ poprawka błędu corr przyjmie stałą wartość, co wywoła sztuczny przeskok lub zamrożenie błędu na brzegach.
- Wolny czujnik ma rzadkie próbki. Interpolacja liniowa tworzy odcinki proste pomiędzy próbkami. Na ostra zakrętach wyinterpolowana pozycja z czujnika wolnego będzie ścinać zakręt (skracając promień), co wytworzy sztuczny błąd pozycji w stosunku do gęstego czujnika.

3.8.5 Problemy

Największym napotkanym problemem w tym zadaniu stał się dla mnie brak wiedzy domenowej i poświęcenie sporej ilości czasu na jej uzupełnianie, np. szukanie możliwych powodów błędnego działania sensorów.